
JFrog Artifactory

Repository Retention Management

Script Reference; Operations Guide

Version	1.0
Dependencies	bash, curl, jq

1. Overview

This document describes three shell scripts used to manage repository retention policies in JFrog Artifactory. Together they form a lightweight workflow for identifying old repositories and applying lifecycle cleanup policies.

Scripts at a Glance

Script	Purpose	Output
GetRepositoryByCreationTime.sh	Inventory all LOCAL repos with age; storage metrics	3 CSV files + terminal summary
CreateRetentionPolicy.sh	Create a new Lifecycle cleanup policy via API	HTTP response + policy JSON
UpdateRetentionPolicy.sh	Update existing policy with a new repo list	HTTP response

Important: Single-Node Artifactory Considerations

IMPORTANT: Since Artifactory is running as a single node, applying a retention policy across many repositories simultaneously can put a significant load on the instance. It is strongly recommended to include only **ONE** repository per policy at a time (Can then gradually increase based on the monitoring), and use `UpdateRetentionPolicy.sh` to cycle through repositories one by one.

2. Recommended Workflow

The three scripts are designed to be used in sequence:

Step	Script	Action
1	GetRepositoryByCreationTime.sh	Run to generate a full CSV inventory. Use the age-filtered CSV to identify repos older than your threshold (e.g. 2 years).
2	policy.json	Edit policy.json to set packageType, retention criteria, and the first repo name.
3	CreateRetentionPolicy.sh	Create the policy with ONE repository in the repos list (single-node best practice).
4	UpdateRetentionPolicy.sh	After the first run completes and results are verified, update the policy with the next repository from your filtered CSV.
5	-	Repeat Step 4 for each repository, monitoring disk usage and Artifactory logs between each cycle.

3. GetRepositoryByCreationTime.sh

Queries the Artifactory storage summary API to build a complete inventory of all LOCAL repositories, enriched with creation timestamps fetched per-repository. Outputs CSV files ready for analysis.

3.1 Prerequisites

- bash 3.2+ (macOS default or Linux)
- curl ; for API calls
- jq ; for JSON parsing
- Bearer token

3.2 Configuration

Edit the top of the script before running:

```
ARTIFACTORY_URL="https://your-instance.jfrog.io/artifactory"  
TOKEN="your-bearer-token-here"
```

3.3 Usage

```
./GetRepositoryByCreationTime.sh
```

The script prompts for an optional age filter:

```
How many years old? (e.g. 6, or press Enter to skip):
```

Enter a number to also generate a filtered CSV of repos older than that many years. Press Enter to skip.

3.4 Output Files

File	Contents
jfrog_local_repos_<timestamp>.csv	Full list of all LOCAL repos with all columns
jfrog_local_repos_older_than_Nyrs_<timestamp>.csv	Filtered list ; only repos older than N years
jfrog_packagetype_summary_<timestamp>.csv	Repos grouped by package type with used space

3.5 CSV Columns

Column	Description
Repo Name	Repository key (unique identifier in Artifactory)
Repo Type	Always LOCAL for this report
Package Type	e.g. Maven, Docker, Npm, Generic, PyPI
Used Space	Human-readable storage used (KB / MB / GB)
Artifact Count	Number of files in the repository
Created Date	Human-readable creation timestamp
Age (Years)	Integer age calculated from creation date to today

Sample Output:

```
sumanthsv@sumanthsv-mac RetentionPolicy % ./test3.sh
```

JFrog Local Repository Report

Filter by age (optional):

Enter the number of years to filter repos older than that age.
Press **Enter** to skip and generate the full report only.

How many years old? (e.g. 6, or press Enter to skip):

✓ No age filter applied. Generating full report only.

🕒 Fetching storage info from Artifactory... please wait.

✓ Storage info fetched successfully (HTTP 200).

Found 499 LOCAL repositories. Fetching creation dates...

[27/499] Fetching details for: anilkt-demo-util-tfstate

jfrog_local_repos_20260614_233130

Repo Name	Repo Type	Package Type	Used Space	Artifact Count	Created Date	Age (Years)
israelrailway-docker-dev-local	LOCAL	Docker	239.02 MB	51	March 18, 2026 07:33 AM UTC	0
jk-maven-local-libs-release-local	LOCAL	Maven	0 bytes	0	April 21, 2026 07:25 AM UTC	0
worksafeproject2-build-info	LOCAL	BuildInfo	0 bytes	0	February 27, 2026 03:03 AM UTC	0
cgsbluecherry-npm-dev-local	LOCAL	npm	7.47 KB	3	November 13, 2025 04:20 PM UTC	0
pypibuild-local-dep	LOCAL	Pypi	0 bytes	0	October 23, 2025 10:19 AM UTC	0
gunvor-docker-local	LOCAL	Docker	35.24 MB	28	April 27, 2026 08:14 AM UTC	0
rbdemo-dev-npm-local	LOCAL	npm	6.96 KB	6	August 14, 2025 06:52 AM UTC	0
temasek-pypi-local	LOCAL	Pypi	0 bytes	0	January 23, 2026 05:43 AM UTC	0
cgsbluecherry-mvn-dev-local	LOCAL	Maven	4.05 KB	3	November 13, 2025 04:19 PM UTC	0
testpromote	LOCAL	Docker	7.62 MB	9	February 25, 2025 06:22 PM UTC	1
blr-npm-dev-remote-local	LOCAL	npm	0 bytes	0	April 21, 2025 03:45 PM UTC	1
rbpromotion-maven-dev-local	LOCAL	Maven	2.07 KB	1	March 30, 2026 06:45 AM UTC	0
dockerrb-local-dev	LOCAL	Docker	7.62 MB	9	February 25, 2025 06:04 PM UTC	1
compare_repo	LOCAL	Generic	601.30 KB	3	June 19, 2025 07:12 AM UTC	0

Generating package type summary report...

Package Type Breakdown:

Package Type	Repos	Artifacts	Total Size
BuildInfo	36	398	8.50 MB
Conan	2	0	0.00 KB
Debian	18	1950	4.62 GB
Docker	132	2441	126.17 GB
Generic	85	19927	65.92 GB
Go	2	5	4.42 MB
Gradle	5	4	487.56 KB
Helm	2	3	0.14 KB
HelmOCI	1	1	1.98 KB
HuggingFaceML	1	0	0.00 KB
MachineLearning	1	0	0.00 KB
Maven	61	1406	439.79 MB
npm	68	572	5.67 MB
NuGet	18	582	223.86 MB
OCI	17	154	1.11 GB
Pypi	42	1762	564.80 MB
ReleaseBundles	6	568	97.06 MB
TerraformBackend	2	2	19.41 KB

Package type summary saved to: jfrog_packagetype_summary_20260614_233858.csv

3.6 Performance Note

The script makes one API call per repository (api/storage/<repo>) to retrieve the creation timestamp. With 400+ repos and a 0.2s sleep between calls, expect a runtime of approximately 2 minutes. This is intentional to avoid overloading a single-node Artifactory instance.

4. CreateRetentionPolicy.sh

Creates a new Artifactory Lifecycle retention policy via the REST API. Reads the policy template from policy.json and injects a list of target repositories from a text file.

4.1 Usage

```
./CreateRetentionPolicy.sh <artifactory_url> <bearer_token> [repos_file]

# Example:
bash CreateRetentionPolicy.sh \
  https://test.jfrog.io \
  cmVmdGtuOjAxO... \
  repos.txt
```

4.2 Parameters

Parameter	Required	Default	Description
artifactory_url	Yes	;	Artifactory base URL (without /artifactory)
bearer_token	Yes	;	Identity token or Access token
repos_file	Yes	repos.txt	Text file with one repo name per line

4.3 Required Files

- policy.json ; The policy template (see Section 6). The script injects the repo list into searchCriteria.repos before POSTing.
- repos.txt ; One repository name per line. For single-node Artifactory, this should contain exactly ONE repo at a time.

```
# repos.txt -- single-node best practice: one repo at a time
dockerrb-local-dev
```

4.4 API Endpoint

```
POST <artifactory_url>/artifactory/api/cleanup/packages/policies/<policy_name>
```

The policy name is hardcoded as POLICY_NAME in the script. Edit this variable if you need a different policy name.

5. UpdateRetentionPolicy.sh

Updates an existing retention policy with a new set of repositories. Identical in structure to CreateRetentionPolicy.sh but uses HTTP PUT instead of POST. Use this to cycle through repositories one at a time on a single-node instance.

5.1 Usage

```
./UpdateRetentionPolicy.sh <artifactory_url> <bearer_token> [repos_file]

# Example -- update to next repo:
bash UpdateRetentionPolicy.sh \
  https://psblr.jfrog.io \
  cmVmdGtuOjAxO... \
  repos.txt
```

5.2 Parameters

Parameter	Required	Default	Description
artifactory_url	Yes	;	Artifactory base URL (without /artifactory)
bearer_token	Yes	;	Identity token or Access token
repos_file	Yes	repos.txt	Text file with one repo name per line

5.3 Difference from Create

Attribute	CreateRetentionPolicy.sh	UpdateRetentionPolicy.sh
HTTP Method	POST	PUT
Use when	Policy does not exist yet	Policy already exists
Effect	Creates a new policy	Overwrites existing policy

6. policy.json

The policy template defines the cleanup criteria. Both Create and Update scripts read this file and inject the repos array before calling the API.

```
{
  "key": "sum-fortescue2",
  "description": "Delete docker packages older than 2 years",
  "itemType": "package",
  "durationInMinutes": 10,
  "enabled": false,
  "skipTrashcan": false,
  "searchCriteria": {
    "packageTypes": ["docker"],
    "repos": ["sumdocker-local"],      <- injected by script from repos.txt
    "createdBeforeInMonths": 24,       <- packages created > 2 years ago
    "lastDownloadedBeforeInMonths": 24, <- AND not downloaded in > 2 years
    "keepLastNVersions": 0
  }
}
```

6.1 Key Fields

Field	Description
key	Unique policy name ; must match POLICY_NAME in scripts
skipTrashcan	false = deleted items go to Trash; true = permanent delete
packageTypes	Array of package types this policy targets
repos	Injected by script ; do not edit manually
createdBeforeInMonths	packages created more than N months ago
lastDownloadedBeforeInMonths	packages not downloaded in more than N months
durationInMinutes	Max runtime per execution (10 minutes, Increase accordingly)

7. End-to-End Example

Step 1 ; Get inventory and identify old repos

```
./GetRepositoryByCreationTime.sh  
# Enter: 2 (filter repos older than 2 years)  
# Output: jfrog_local_repos_older_than_2yrs_20260612.csv
```

Step 2 ; Pick the repo and create repos.txt

```
echo 'dockerrb-local-dev' > repos.txt
```

Step 3 ; Create the policy (first time only)

```
bash CreateRetentionPolicy.sh \  
  https://psblr.jfrog.io \  
  cmVmdGtuOjAxO...  
  
# Output: Policy created successfully (HTTP 200)
```

Step 4 ; Monitor, then move to next repo

```
# Check Artifactory logs and disk usage, then:  
echo 'newtest-docker-local' > repos.txt  
  
bash UpdateRetentionPolicy.sh \  
  https://psblr.jfrog.io \  
  cmVmdGtuOjAxO...  
  
# Output: Policy updated successfully (HTTP 200)
```

Repeat Step 4 for every repository in your filtered CSV. There is no need to delete and recreate the policy ; UpdateRetentionPolicy.sh handles successive updates.